Lesson: Linear Regression and Regularization

Introduction

In our previous lesson, we explored classification models—Logistic Regression, Decision Trees, and k-NN—where the question was, "*What category does this data point belong to?*" In this lesson, we shift from classification to **prediction**. Now, we ask: **"How much?"** or **"What value?"**

Imagine predicting house prices based on square footage and location, or estimating future sales based on past performance. These are classic use cases for regression models, especially Linear Regression. But as you'll soon discover, not all regression lines are created equal. Models can underfit or overfit—just like in classification—and that's where regularization techniques come to the rescue

Learning Outcomes

By the end of this lesson, you will be able to:

- Understand linear and polynomial regression models.
- Detect underfitting and overfitting in regression.
- Apply regularization techniques: Ridge and Lasso Regression.

Introduction to Linear Regression

Linear Regression is one of the simplest and most widely used techniques for modeling the relationship between a dependent variable and one or more independent variables. It is the preferred approach for activities like sales forecasting, housing price prediction, and more since it is especially effective at predicting numerical quantities.

Assumptions:

- Linearity: The relationship between input and output is linear.
- Homoscedasticity: Constant variance of errors.
- No multicollinearity: Features should not be too correlated.
- Normally distributed residuals.

Code Snippet:

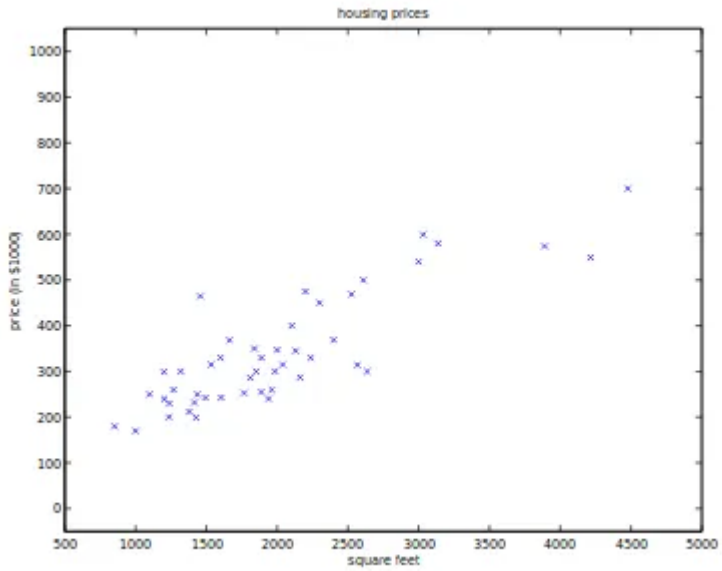
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Example:

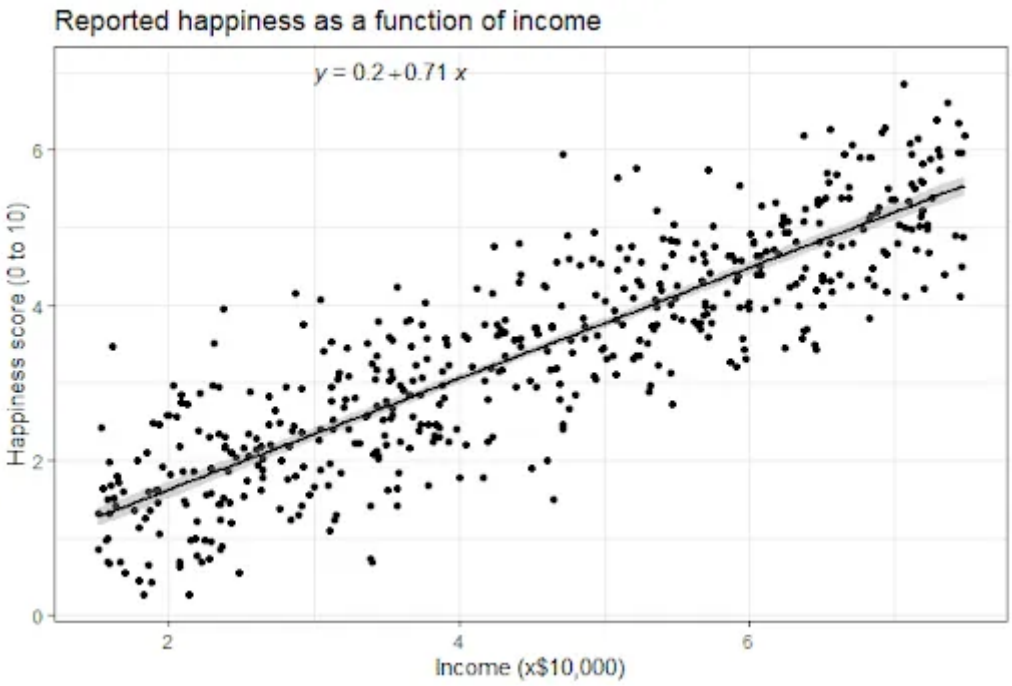
Let's meet **Joe**. He is a trying to buy a house and is collecting housing data so that he can estimate the "cost" of the house according to the "Living area" of the house in feet.

Living area (feet^2)	**Price (1000$s)**
2104	400
1600	330
2400	369
1416	232
3000	540
⋮	⋮

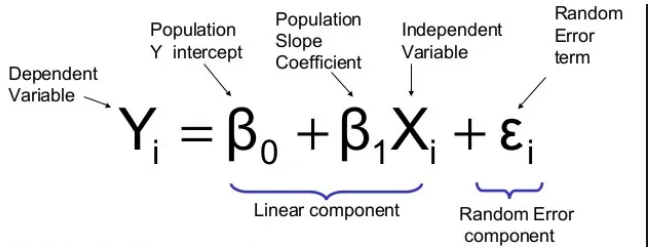
Joe plots the data and notices a linear pattern. For his first scatter plot, joe uses two variables: 'Living area' and 'Price'.



The idea? **Find a line that best fits the data** minimizing the difference between actual and predicted values like in the below figure.



Linear regression equation:



Where:

- Y is the dependent variable you want to predict.
- X1 , X2 , … , Xn are the independent variables.
- b0 is the intercept (the predicted value of Y when all X are zero).
- b1 , b2 , …,bn are the coefficients that represent the change in Y for a one-unit change in X1,X2,…,Xn while holding all other variables constant.

To make accurate predictions, we must choose the best values for b0 and b1. How?

Cost Function and Gradient Descent

We use a **cost function** to measure prediction error. The most common one is **Mean Squared Error (MSE)**:

Error = Σ (actual output — predicted output)^2

$$J(\theta) = \frac{1}{2} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

To minimize this error, we use **Gradient Descent** an iterative optimization technique that updates parameters to reduce the cost.

Introduction to Polynomial Regression

While Linear Regression works well when the relationship between the variables is approximately linear, there are cases where a straight line is too simplistic to capture the underlying patterns. Polynomial Regression, on the other hand, is an extension of Linear Regression that can capture more complex, nonlinear relationships.

How Does It Work?

The core idea is to transform the original input features into a higher-degree polynomial space and then apply linear regression on these transformed features.

It still uses Ordinary Least Squares (OLS) to find the best-fitting curve, but thanks to the feature transformation, the model can now fit curves instead of straight lines.

The degree of the polynomial determines the flexibility of the model:

- Degree = 1 → Linear
- Degree = 2 → Quadratic
- Degree = 3 → Cubic, and so on.

ParseError: KaTeX parse error: Can't use function '$' in math mode at position 56: … cdots\beta_nx^n$$![Polynomia…

\text{Loss} = \text{MSE} + \lambda \sum_{j=1}^{n} w_j^2

ParseError: KaTeX parse error: Can't use function '$' in math mode at position 8: where λ is the…

\text{Loss} = \text{MSE} + \lambda \sum_{j=1}^{n} |w_j|

ParseError: KaTeX parse error: Can't use function '$' in math mode at position 312: …tion parameter λ. ![Re…